Karta projektu - Bezpieczeństwo Aplikacji i Systemów Mobilnych

Szablon do wypełnienia przez sekcję na zajęciach 1. Finalną wersję umieszczacie w README repozytorium przed zajęciami 2. Karta jest punktem odniesienia przez cały semestr: zmiany zgłaszacie prowadzącemu.

Informacje o sekcji

Pole	Wartość
Nazwa sekcji / aplikacji	Sekcja: cybezpieczniki (podsekcja 5, grupa 2) Aplikacja: **S-Bank** - prosta bankowość mobilna i webowa (nazwa robocza)
Technologia	**React Native** (Expo, TypeScript) - jedna baza kodu na Android, iOS i przeglądarkę (react-native-web)
Skład sekcji (imię i nazwisko)	Michał Pokrzywa, Mateusz Falfus, Kacper Sikorski
Link do repozytorium	
Backend	**Własny**

1. Koncepcja aplikacji

Opiszcie aplikację w maksymalnie trzech zdaniach: co robi, dla kogo i jaki problem rozwiązuje.

S-Bank to bardzo prosta aplikacja bankowa (mobilna i webowa) dla klienta indywidualnego: pokazuje rachunki i historię operacji, pozwala wykonać przelew - także z kodu QR lub linku płatności - i przelicza saldo na waluty obce po kursach NBP. Bank wystawia też publiczne API: otwarte kursy walut oraz interfejs dla partnerów (np. sklepów internetowych), którzy tworzą żądania płatności. Aplikacja jest celowo mała funkcjonalnie, bo jej zadaniem jest pokazanie na działającym przykładzie poprawnie skonfigurowanych, gotowych mechanizmów ochrony: OpenID Connect z PKCE, passkeys ze step-up, mTLS i PKCS#11 oraz kontroli dostępu opartej na rolach i własności zasobu.

Grupa docelowa: klienci indywidualni korzystający z bankowości w telefonie i w przeglądarce; pracownicy obsługi klienta i administrator banku (panel w tej samej aplikacji); partnerzy integrujący się z bankiem przez API.

Temat startowy, z którego korzystacie (jeśli dotyczy): temat własny - prosta bankowość mobilna. *wpisać numer/nazwę tematu, jeśli na liście prowadzącego jest odpowiednik*

2. Role użytkowników

Wymagane są co najmniej dwie role. Kolumna „Czego NIE może” jest równie ważna jak „Co może” - to z niej wyjdą testy autoryzacji.

Rola	Co może	Czego NIE może
Klient (customer)	• logować się hasłem lub passkey • widzieć wyłącznie własne rachunki, saldo i historię • zlecać przelewy z własnych rachunków w ramach limitu dziennego (domyślnie 5 000 zł) • zlecać przelewy od 1 000 zł po dodatkowym potwierdzeniu passkey (step-up) • zarządzać własną listą odbiorców • opłacać żądania płatności z kodu QR lub linku • włączyć odblokowanie aplikacji biometrią	• odczytać rachunku, historii ani odbiorców innego klienta - także przez podmianę identyfikatora w żądaniu do API • zlecić przelewu z cudzego rachunku • przekroczyć limitu dziennego ani samodzielnie go zmienić • zlecić przelewu od 1 000 zł bez świeżego step-up • zablokować ani odblokować rachunku • wejść do panelu pracownika i administratora ani czytać dziennika audytowego • wywołać API partnerskiego

Rola	Co może	Czego NIE może
Pracownik banku (employee)	• wyszukiwać klientów • podglądać rachunki i historię klienta w trybie tylko do odczytu (każdy podgląd trafia do dziennika audytowego) • blokować i odblokowywać rachunek • zmieniać limit dzienny klienta w dozwolonym przedziale	• zlecić ani zatwierdzić przelewu w imieniu klienta • zmienić salda, edytować ani usunąć transakcji • nadawać ról ani tworzyć kont pracowników • czytać ani czyścić dziennika audytowego • zarządzać partnerami API i ich certyfikatami • zalogować się samym hasłem (wymagany passkey)
Administrator (admin)	• zarządzać kontami i rolami (Keycloak) • rejestrować partnerów API i wystawiać im certyfikaty • czytać dziennik audytowy • ustawiać parametry globalne (próg step-up, zakres limitów)	• zlecić przelewu z rachunku klienta ani zmienić salda • modyfikować ani usuwać wpisów dziennika audytowego (tabela tylko do dopisywania) • zobaczyć haseł, kluczy passkey ani klucza prywatnego w HSM • zalogować się samym hasłem (wymagany passkey)
Partner API (partner, klient maszynowy)	• uwierzytelnić się certyfikatem klienta (mTLS) i pobrać token • utworzyć żądanie płatności (kwota, tytuł) na własny rachunek • sprawdzić status własnych żądań płatności	• odczytać salda, historii ani danych osobowych klienta • doprowadzić do przelewu bez zatwierdzenia przez klienta w aplikacji • zobaczyć żądań płatności innego partnera • użyć tokenu z innym certyfikatem niż ten, na który token wydano • połączyć się z /partner bez certyfikatu podpisanego przez nasze CA
Gość (niezalogowany)	• czytać kursy walut z publicznego API • otworzyć regulamin i listę bankomatów	• wywołać żadnego innego endpointu - API domyślnie odmawia (deny by default) • przekroczyć limitu liczby zapytań (rate limiting)

3. Dane i ich klasyfikacja

Wypiszcie wszystkie dane, które aplikacja zbiera, przetwarza lub przechowuje. Przy każdej pozycji określcie poziom wrażliwości.

Dane	Skąd pochodzą	Gdzie są przechowywane (urządzenie / backend / oba)	Klasyfikacja	Uzasadnienie
Dane profilu: imię i nazwisko, e-mail	użytkownik przy rejestracji	Backend (baza Keycloak). Urządzenie: tylko w pamięci (z ID tokenu)	Poufne	Dane osobowe; e-mail jest loginem
Hasło (metoda zapasowa obok passkey)	użytkownik	Backend - Keycloak, wyłącznie skrót hasła. Aplikacja nigdy go nie widzi (logowanie w przeglądarce systemowej)	Poufne	Wyciek skrótów naraża konta także w innych serwisach
Passkey: klucz publiczny i identyfikator poświadczenia	generowane na urządzeniu przy rejestracji passkey	Klucz publiczny: backend (Keycloak). Klucz prywatny: bezpieczny magazyn systemu - poza zasięgiem aplikacji	Poufne	Klucz publiczny nie jest sekretem, ale jest powiązany z kontem; prywatnego nie przetwarzamy
Access token (JWT, ważny 5 min)	Keycloak	Urządzenie: tylko pamięć RAM. Backend go nie zapisuje - weryfikuje podpis	Poufne	Daje dostęp do API do chwili wygaśnięcia
Refresh token	Keycloak	Urządzenie: expo-secure-store (Keychain / Keystore) z wymogiem biometrii. Web: tylko pamięć	Poufne	Długo żyjący - kradzież oznacza przejęcie sesji; rotowany przy każdym użyciu

Dane	Skąd pochodzą	Gdzie są przechowywane (urządzenie / backend / oba)	Klasyfikacja	Uzasadnienie
Numer rachunku i saldo	backend (rachunek demonstracyjny tworzony przy rejestracji)	Backend (PostgreSQL). Urządzenie: tylko pamięć, bez zapisu na dysk	Szczególnie wrażliwe	Dane finansowe; minimalizacja - brak kopii lokalnej
Historia transakcji	powstaje z przelewów	Backend. Urządzenie: tylko pamięć	Szczególnie wrażliwe	Finanse; pozwala odtworzyć zwyczaje i kontakty klienta, dlatego poziom wyższy niż minimalny
Lista zapisanych odbiorców	użytkownik	Backend	Poufne	Dane osób trzecich (nazwa, numer rachunku)
Żądania płatności partnerów (kwota, tytuł, status)	Partner API	Backend	Poufne	Dane handlowe partnera powiązane z klientem
Dokładna lokalizacja	GPS urządzenia, za zgodą	Tylko urządzenie, tylko pamięć. Nie jest wysyłana do backendu	Szczególnie wrażliwe	Służy wyłącznie do posortowania bankomatów według odległości - liczone lokalnie
Obraz z kamery (skan QR)	kamera, za zgodą	Tylko urządzenie, przetwarzany w locie; klatki nie są zapisywane	Wewnętrzne	Z kodu odczytujemy jedynie identyfikator żądania płatności
Dane biometryczne	czytnik linii papilarnych / rozpoznawanie twarzy	System operacyjny. Aplikacja dostaje tylko wynik: zgodne / niezgodne	Szczególnie wrażliwe	Nie przetwarzamy i nie przechowujemy - wzorzec zostaje w systemie
Preferencje: motyw, język, flaga „biometria włączona”	użytkownik	Urządzenie (AsyncStorage)	Wewnętrzne	Brak danych osobowych; wystarcza izolacja piaskownicy aplikacji
Dziennik audytowy: kto, co, kiedy, adres IP, poziom uwierzytelnienia	backend	Backend (tabela tylko do dopisywania)	Poufne	Zawiera identyfikatory użytkowników i adresy IP
Klucz podpisujący potwierdzenia przelewów; klucz naszego CA dla mTLS	generowane wewnątrz tokenu	Backend - SoftHSM (PKCS#11), klucz nieeksportowalny	Poufne	Kompromitacja pozwala fałszować potwierdzenia i wystawiać certyfikaty partnerów
Certyfikaty partnerów (publiczne) i ich odciski	wystawia nasze CA	Backend	Wewnętrzne	Klucz prywatny zostaje u partnera
Kursy walut NBP, lista bankomatów, regulamin	NBP Web API / dane własne	Backend (cache); urządzenie (pamięć)	Publiczne	Dane jawne

Poziomy klasyfikacji:

Poziom	Przykłady	Minimalna ochrona
Publiczne	opis wydarzenia, cennik, regulamin	brak szczególnych wymagań
Wewnętrzne	preferencje, historia wyszukiwań, ustawienia	izolacja od innych aplikacji
Poufne	e-mail, tokeny sesji, historia transakcji	bezpieczny magazyn systemowy, szyfrowanie

Poziom	Przykłady	Minimalna ochrona
Szczególnie wrażliwe	zdrowie, biometria, finanse, dokładna lokalizacja	minimalizacja danych; często brak przechowywania lokalnego

4. Mapowanie na umowę o powierzchni ataku

Każdy projekt musi zawierać wszystkie poniższe funkcje. Opiszcie, jak konkretnie pojawią się w waszej aplikacji.

Wymagana funkcja	Jak realizujemy ją w naszej aplikacji	Planowane na zajęcia
Logowanie z tokenami i co najmniej dwie role	• OpenID Connect: Authorization Code + PKCE do Keycloak przez przeglądarkę systemową (expo-auth-session) - aplikacja nie dotyka hasła • access token JWT ważny 5 min i rotowany refresh token • role w tokenie: customer, employee, admin oraz klient maszynowy partner • API sprawdza podpis (JWKS), wystawcę, odbiorcę i ważność tokenu biblioteką jose • logowanie passkey; przelew od 1 000 zł wymaga step-up (poziom acr sprawdzany także w API)	Z3 (OIDC, role), Z4 (passkey, step-up)
Własny backend/API lub Firebase/Supabase z regułami dostępu	• własny backend – reguły dostępu zapisujemy w kodzie API, a nie w regułach bazy • własne REST API (NestJS) za reverse proxy Caddy (TLS) • domyślna odmowa: globalny strażnik wymaga tokenu, każdy endpoint deklaruje rolę, a polityki CASL sprawdzają właściciela rekordu • trzy strefy: /public (bez logowania, limit zapytań), /api (token użytkownika), /partner (mTLS + token związany z certyfikatem) • walidacja wejścia, dokumentacja OpenAPI, dziennik audytowy	Z2 (szkielet), Z3 (autoryzacja), Z5 (mTLS)
Lokalne przechowywanie danych wrażliwych (tokeny, dane osobowe, cache)	• refresh token w expo-secure-store (Keychain / Keystore) z opcją requireAuthentication • access token i dane finansowe tylko w pamięci; czyszczone przy wylogowaniu i po bezczynności • preferencje w AsyncStorage; żadnych danych wrażliwych w logach • zasłonięty podgląd w przełączniku aplikacji; na web tokeny tylko w pamięci	Z3
Co najmniej trzy uprawnienia systemowe (kamera, lokalizacja, pliki, BLE)	• **kamera** - skan kodu QR z żądaniem płatności (expo-camera) • **lokalizacja** (tylko podczas używania) - najbliższe bankomaty (expo-location) • **powiadomienia** - lokalne potwierdzenie wykonania przelewu (expo-notifications, bez push przez Firebase Cloud Messaging) • o zgodę prosimy dopiero w chwili użycia funkcji; po odmowie aplikacja działa dalej	Z4 (kamera), Z5 (lokalizacja, powiadomienia)
Deep link lub universal link	• universal / app link https://<domena>/pay/{id} (zapasowo sejfik://pay/{id}) otwiera ekran potwierdzenia płatności • link niesie wyłącznie identyfikator - kwotę i odbiorcę aplikacja pobiera z API; wymagane logowanie i jawne potwierdzenie • drugi link: powrót z logowania OIDC, chroniony przez PKCE i parametr state	Z4
WebView lub wczytywanie treści zewnętrznych	• regulamin i pomoc z własnej domeny w react-native-webview • lista dozwolonych adresów, tylko HTTPS, wyłączony JavaScript i dostęp do plików, brak mostu JS-natywny • linki zewnętrzne otwierane w przeglądarce systemowej; logowanie celowo NIE odbywa się w WebView	Z5

Wymagana funkcja	Jak realizujemy ją w naszej aplikacji	Planowane na zajęcia
Biometria jako opcja odblokowania	• expo-local-authentication: odblokowanie po uruchomieniu i po powrocie z tła • refresh token w SecureStore jest związany z biometrią, więc samo ominięcie ekranu blokady nie daje sesji • funkcja opcjonalna - zawsze dostępne jest pełne logowanie	Z4

5. Architektura

Wymieńcie główne komponenty i połączenia między nimi. Wstępny szkic (zdjęcie z tablicy też się liczy) dołączcie do repozytorium - na zajęciach 2 zamienimy go w diagram przepływu danych.

Komponent	Technologia	Z czym się komunikuje
Aplikacja mobilna (i webowa)	React Native + Expo (TypeScript), Expo Router, react-native-web, TanStack Query	• Keycloak - logowanie OIDC w przeglądarce systemowej (HTTPS) • Backend API - HTTPS, nagłówek Bearer z access tokenem
Reverse proxy (dodatkowy komponent)	Caddy: TLS 1.3, certyfikaty Let's Encrypt, weryfikacja certyfikatu klienta dla /partner	• przyjmuje cały ruch z internetu • przekazuje go do API i Keycloak w sieci wewnętrznej
Backend / API	Node.js + TypeScript, NestJS; jose (JWT), CASL (polityki dostępu), class-validator, helmet, throttler; OpenAPI	• PostgreSQL (Prisma) • Keycloak - pobranie kluczy publicznych (JWKS) • SoftHSM - podpisy przez PKCS#11 • NBP Web API - kursy walut (HTTPS, z cache)
Baza danych	PostgreSQL + Prisma (zapytania parametryzowane); osobna baza dla Keycloak	• wyłącznie backend, w sieci wewnętrznej Dockera • brak dostępu z internetu
Usługi zewnętrzne (logowanie, płatności, mapy, analityka, push…)	• **Keycloak** (u nas) - OIDC, passkeys, role, step-up • **SoftHSM2** (u nas) - magazyn kluczy PKCS#11 • **NBP Web API** - publiczne kursy walut, bez klucza • Let's Encrypt - certyfikaty serwera • bez Firebase / Supabase, analityki i zewnętrznych SDK	• Keycloak: aplikacja i API • SoftHSM: tylko backend • NBP: tylko backend - aplikacja nie łączy się z nim bezpośrednio

6. Podział ról w zespole

Osoba	Główna odpowiedzialność	Odpowiedzialność za bezpieczeństwo (obszar MASVS)
Michał Pokrzywa	Aplikacja React Native: ekrany, nawigacja, wersja webowa	MASVS-STORAGE, MASVS-PLATFORM (deep linki, WebView, uprawnienia)
Mateusz Falfus	Backend API, baza danych, integracja z NBP, OpenAPI	Autoryzacja po stronie serwera (role i własność zasobu), MASVS-CODE (walidacja wejścia, zależności)
Kacper Sikorski	Infrastruktura: Docker, Caddy, CA i mTLS, SoftHSM, CI, testy bezpieczeństwa Tożsamość: Keycloak, OIDC, passkeys, step-up, biometria	MASVS-NETWORK, MASVS-CRYPTO, MASVS-PRIVACY, MASVS-AUTH

7. Plan dojścia do MVP (zajęcia 5)

Zajęcia	Co będzie gotowe
Z2	Szkielet aplikacji, który się buduje; szkic architektury. Konkretnie: aplikacja Expo startuje na Androidzie i w przeglądarce, API odpowiada na /health, Docker Compose podnosi PostgreSQL i Keycloak, karta w README.
Z3	Logowanie OIDC z PKCE i role w tokenie; API weryfikuje JWT; rachunki i historia z kontrolą własności zasobu; refresh token w SecureStore; publiczny endpoint kursów NBP; pierwsze testy autoryzacji z kolumny „Czego NIE może”.
Z4	Przelewy z limitem dziennym; passkey i step-up dla przelewów od 1 000 zł; odblokowanie biometrią; żądanie płatności przez deep link i kod QR (kamera); panel pracownika.
Z5 - MVP	Działająca aplikacja ze wszystkimi funkcjami z umowy (mogą być uproszczone). Dochodzą: API partnerskie z mTLS, WebView z regulaminem, bankomaty (lokalizacja), powiadomienia, panel administratora z dziennikiem audytowym, podpis potwierdzeń przez PKCS#11 w wersji podstawowej.
Po MVP	Pełny opis działania mechanizmów w repozytorium, komplet testów autoryzacji, przegląd według MASVS, poprawki po testach.

8. Największe ryzyko według sekcji

Co według was zagrożone jest w pierwszej kolejności i dlaczego? Wpiszcie 1-3 pozycje. Na koniec semestru sprawdzimy, czy mieliście rację.

1. **Złamana kontrola dostępu do obiektów w API (BOLA / IDOR).** Klient podmienia identyfikator rachunku lub transakcji w żądaniu i czyta cudze dane albo zleca przelew z cudzego rachunku. To najbardziej prawdopodobny błąd, bo mamy cztery role, trzy strefy API i dane finansowe, a wystarczy jeden endpoint bez sprawdzenia właściciela. Ograniczamy to domyślną odmową, politykami w jednym miejscu i testami generowanymi z kolumny „Czego NIE może”.
2. **Kradzież lub nadużycie tokenów i przechwycenie linków.** Refresh token na urządzeniu, tokeny w wersji webowej (XSS), przechwycenie powrotu z logowania OIDC przez inną aplikację oraz spreparowany link płatności z podmienioną kwotą. Ograniczamy to przez PKCE, bezpieczny magazyn związany z biometrią, krótkie życie tokenów z rotacją i linki niosące tylko identyfikator.
3. **Błędna konfiguracja gotowych klocków.** Budujemy własny backend z Keycloak, Caddy i HSM, bez zarządzanej usługi typu Firebase, więc całe ryzyko konfiguracji jest po naszej stronie: zbyt szerokie adresy przekierowań, tryb deweloperski i domyślne hasła, klucze prywatne w repozytorium, /partner dostępne z pominięciem mTLS, step-up egzekwowany tylko po stronie Keycloak. Dlatego API samo sprawdza poziom acr i odcisk certyfikatu, sekrety są poza repozytorium, a konfiguracja jest w kodzie i przechodzi przegląd.

Dodatek. Mechanizmy ochrony pokazywane w projekcie

Sekcja dodana przez nas - wymaganie projektu to minimum trzy mechanizmy zbudowane z gotowych, zaufanych bibliotek. Nie implementujemy własnej kryptografii ani własnych protokołów; szczegółowy opis działania każdego mechanizmu trafi do repozytorium.

Mechanizm	Gdzie w aplikacji	Jak działa (skrót)	Gotowe klocki
1. OpenID Connect - Authorization Code + PKCE	Logowanie w aplikacji mobilnej i webowej	Aplikacja losuje sekret (code_verifier) i wysyła do Keycloak jego skrót. Użytkownik loguje się na stronie Keycloak w przeglądarce systemowej. Aplikacja dostaje jednorazowy kod i wymienia go na tokeny, pokazując sekret - przechwycony kod jest bez niego bezużyteczny. API sprawdza podpis tokenu kluczem publicznym Keycloak.	Keycloak, expo-auth-session, expo-web-browser, jose

Mechanizm	Gdzie w aplikacji	Jak działa (skrót)	Gotowe klocki
2. Passkeys (WebAuthn / FIDO2) + step-up	Logowanie bez hasła; potwierdzenie przelewu od 1 000 zł; obowiązkowe dla pracownika i administratora	Urządzenie tworzy parę kluczy przypisaną do domeny banku; serwer zna tylko klucz publiczny. Przy logowaniu urządzenie po biometrii podpisuje wyzwanie serwera - na fałszywej domenie podpis nie powstanie. Step-up: aplikacja prosi o wyższy poziom (acr_values), Keycloak wymaga passkey i wydaje token z acr=2, a API samo sprawdza poziom i świeżość uwierzytelnienia.	Keycloak (WebAuthn Passwordless, poziomy LoA w przepływie logowania)
3. mTLS + token związany z certyfikatem (RFC 8705)	API partnerskie /partner	W uzgadnianiu TLS certyfikat pokazuje także klient; Caddy przyjmuje tylko certyfikaty podpisane przez nasze CA. Keycloak wpisuje do tokenu odcisk certyfikatu (cnf), a API porównuje go z certyfikatem bieżącego połączenia - skradziony token bez klucza prywatnego jest bezużyteczny.	Caddy (client_auth), Keycloak (uwierzytelnianie klienta X.509), step-ca lub OpenSSL jako CA
4. PKCS#11 (HSM)	Podpis potwierdzeń przelewów	Klucz prywatny powstaje wewnątrz tokenu SoftHSM jako nieeksportowalny. Backend otwiera sesję PKCS#11, przekazuje skrót potwierdzenia i odbiera podpis - klucz nigdy nie trafia do pamięci procesu ani na dysk. Potwierdzenie można zweryfikować kluczem publicznym z /public/keys. Sprzętowy HSM używa tego samego interfejsu.	SoftHSM2, graphene-pk11 (pkcs11js)
5. Kontrola dostępu: role + własność zasobu	Każde wywołanie API	Domyślna odmowa. Role pochodzą z tokenu, a polityki zapisane są w jednym miejscu, np. „klient czyta rachunek, którego jest właścicielem”. Cudzy zasób zwraca 404, żeby nie zdradzać jego istnienia. Testy automatyczne odpowiadają wierszom kolumny „Czego NIE może”.	NestJS Guards, CASL, role Keycloak, Jest + supertest
Uzupełniająco	Urządzenie i przeglądarka	Bezpieczny magazyn i biometria na urządzeniu; nagłówki bezpieczeństwa i CSP w wersji webowej; limit zapytań; walidacja wejścia.	expo-secure-store, expo-local-authentication, helmet, @nestjs/throttler

Akceptacja prowadzącego

Data	Status	Uwagi
	zaakceptowano / do poprawy	